

Programlama Dilleri

Dr. Öğr. Üyesi Hayri Volkan Agun

Bilgisayar Mühendisliği Bölümü

Bursa Teknik Üniversitesi

Nesneye Yönelik Kavramlar

- Nesne yönelimli programlama, modern programlama dillerinde çok yaygın hâle gelmiş bir programlama yaklaşımıdır. Günümüzde birçok dil ya doğrudan nesne yönelimli olarak tasarlanmıştır ya da sonradan nesne yönelimli özellikler kazanmıştır.
- Nesne yönelimli programlama artık ana akım bir yaklaşımdır.
- COBOL'dan LISP'e kadar birçok dilin nesne yönelimli sürümleri veya uzantıları ortaya çıkmıştır.
- Dillerin OOP desteği aynı değildir; her dil inheritance, dynamic binding ve class kavramlarını farklı biçimde tasarlayabilir.
- Bu bölümün temel amacı, özellikle **inheritance** ve **dynamic binding** tasarım kararlarını incelemektir.

Nesne Tabanlı Diller

Smalltalk	Saf nesne yönelimli dil modeli
C++	Procedural + object-oriented
Objective-C	C tabanlı object-oriented genişleme
Java	Güçlü object-oriented yapı
C#	Java'ya benzer modern OOP dili
Ruby	Her şeyin nesne olduğu fakat procedural kullanım da sağlayan dil
Ada 95	OOP özellikleri eklenmiş Ada sürümü
CLOS	LISP'in object-oriented uzantısı

Burada önemli olan yalnızca “bir dil OOP destekliyor mu?” sorusu değildir. Daha önemli soru şudur: Dil, inheritance ve dynamic binding gibi özellikleri nasıl tasarlamıştır?

Nesneye Yönelik Programlama

- Nesne yönelimli programlama, soyut veri tipleri üzerine kurulmuş bir genişleme gibi düşünülebilir.
- **Abstract Data Type — ADT nedir?**
- ADT, veriyi ve o veri üzerinde yapılabilecek işlemleri birlikte tanımlar. Kullanıcı, verinin iç temsilini bilmek zorunda değildir.

`</>` Java

```
class Stack {  
    private int[] data;  
    private int top;  
  
    void push(int x) {  
        data[++top] = x;  
    }  
  
    int pop() {  
        return data[top--];  
    }  
}
```

Nesneye Yönelik Programlama

- OOP, benzer abstract data type'lar arasındaki ortaklığı ayırır ve bu ortaklığı yeni bir tipe koyar. Diğer tipler bu ortak kısımları inheritance yoluyla devralır.
- Car, Truck ve Bus farklı sınıflardır; fakat hepsinin ortak özellikleri olabilir:
 - year
 - color
 - make
 - start()
 - stop()
- Bu ortak kısımlar Vehicle sınıfına alınır.

```
Vehicle
├── Car
├── Truck
└── Bus
```

Nesneye Yönelik Programlama

```
</> Java

class Vehicle {
    int year;
    String color;
    String make;

    void start() {
        System.out.println("Vehicle started");
    }
}

class Truck extends Vehicle {
    int numberOfWheels;
    double haulingCapacity;
}
```

- Car, Truck ve Bus farklı sınıflardır; fakat hepsinin ortak özellikleri olabilir:
 - year
 - color
 - make
 - start()
 - stop()
- OOP, ADT'nin encapsulation fikrini korur; fakat buna sınıflar arası ilişki kurma ve ortaklığı yeniden kullanma yeteneği ekler.

Kalıtım (Inheritance)

- Inheritance, metinde iki önemli probleme çözüm olarak sunulur:
 - Yazılım yeniden kullanımı problemi
 - Program organizasyonu problemi
- Var olan bir abstract data type çoğu zaman yeni problem için tam olarak uygun değildir. Küçük değişiklikler gerekebilir. Ancak mevcut kodu değiştirmek zor olabilir çünkü:
 - Kod karmaşık olabilir.
 - Kodu yazan kişi farklı olabilir.
 - Değişiklikler client programları bozabilir.
 - Var olan sınıfın test edilmiş davranışı zarar görebilir.
- Inheritance'ın çözümü: Var olan sınıfı değiştirmek yerine yeni bir subclass oluşturulur.

Java

```
class Vehicle {  
    int year;  
    String color;  
    String make;  
}  
  
class Truck extends Vehicle {  
    double haulingCapacity;  
    int numberOfWheels;  
}
```

Class, Object, Method ve Message Kavramları

Kavram	Açıklama
Class	Nesne türünü tanımlayan yapı
Object	Bir class'ın örneği
Method	Nesne üzerinde çalışan işlem
Message	Nesneye gönderilen metod çağrısı
Superclass	Üst sınıf
Subclass	Alt sınıf
Message Interface	Nesnenin desteklediği metotlar bütünü

Subclass Parent Class'tan Nasıl Farklılaşır?

- Bir subclass, superclass'tan üç temel biçimde farklılaşabilir.
- **Bazı üyeler alt sınıfta görünmeyebilir**
 - Üst sınıfta private tanımlanan değişken veya metotlar alt sınıfta doğrudan erişilebilir olmayabilir.

```
</> Java

class Vehicle {
    private int year;
    protected String make;
}

class Truck extends Vehicle {
    void printMake() {
        System.out.println(make); // erişilebilir
        // System.out.println(year); // doğrudan erişilemez
    }
}
```

Subclass Parent Class'tan Nasıl Farklılaşır?

- Bir subclass, superclass'tan üç temel biçimde farklılaşabilir.
- **Subclass yeni değişken veya metot ekleyebilir.**

`</> Java`

```
class Truck extends Vehicle {  
    double haulingCapacity;  
    int numberOfWheels;  
  
    void loadCargo() {  
        System.out.println("Cargo loaded");  
    }  
}
```

- Truck, Vehicle özelliklerini devralır; ayrıca kendi özelliklerini ekler.

Subclass Parent Class'tan Nasıl Farklılaşır?

- Bir subclass, superclass'tan üç temel biçimde farklılaşabilir.
- Subclass inherited method'u değiştirebilir. Bu işleme **method overriding** denir.
- Burada Waterfowl, Bird sınıfından gelen draw metodunu override eder.
- Bir Bird sınıfı genel bir kuş çizebilir. Waterfowl sınıfı ise aynı draw metodunu su kuşuna özgü biçimde yeniden tanımlayabilir.

```
<> Java

class Bird {
    void draw() {
        System.out.println("Drawing a generic bird");
    }
}

class Waterfowl extends Bird {
    @Override
    void draw() {
        System.out.println("Drawing a waterfowl");
    }
}
```

Dinamik Bağlama

- Nesne yönelimli programlama dillerinin üç temel özelliği vardır:
 - Abstract Data Types
 - Inheritance
 - Dynamic Binding (Dynamic Dispatch) of Method Calls to Methods
- **Dynamic binding nedir?**
 - Bir nesneye gönderilen mesajın veya çağrılan metodun, hangi gerçek metoda karşılık geleceğinin **çalışma zamanında** belirlenmesidir.
 - Yani derleyici, her zaman şu soruya kesin cevap veremez:
 - draw() çağrısı hangi sınıftaki draw() metodunu çalıştıracak?
 - Bu cevap, çalışma zamanındaki gerçek nesne tipine bağlı olabilir.

Dynamic Binding

```
</> Java

A obj;

obj = new A();
obj.draw(); // A'nın draw metodu çalışır

obj = new B();
obj.draw(); // B'nin draw metodu çalışır
```

- A bir base class'tır.
- B, A sınıfının subclass'ıdır.
- B, A sınıfından gelen draw() metodunu override eder.

```
</> Java

class A {
    void draw() {
        System.out.println("Drawing A");
    }
}

class B extends A {
    @Override
    void draw() {
        System.out.println("Drawing B");
    }
}
```

Dynamic Binding

```
</> Java

A obj;

obj = new A();
obj.draw(); // A'nın draw metodu çalışır

obj = new B();
obj.draw(); // B'nin draw metodu çalışır
```

- Buradaki değişkenin tipi A olmasına rağmen ikinci çağrıda B.draw() çalışır.
- Çünkü obj değişkeninin gösterdiği gerçek nesne çalışma zamanında B tipindedir.

```
</> Java

class A {
    void draw() {
        System.out.println("Drawing A");
    }
}

class B extends A {
    @Override
    void draw() {
        System.out.println("Drawing B");
    }
}
```

Dynamic Binding

```
</> Java

A obj;

obj = new A();
obj.draw(); // A'nın draw metodu çalışır

obj = new B();
obj.draw(); // B'nin draw metodu çalışır
```

- Dynamic binding sayesinde aynı çağrı ifadesi farklı nesne tiplerinde farklı davranış üretebilir. Bu satır bazen A.draw(), bazen B.draw() çalıştırabilir.
- Bu özellik, OOP'de polymorphism'in temelidir.

```
</> Java

class A {
    void draw() {
        System.out.println("Drawing A");
    }
}

class B extends A {
    @Override
    void draw() {
        System.out.println("Drawing B");
    }
}
```

Polymorphic Reference

```
Java
```

```
A ref;
```

```
ref = new A(); // geçerli
```

```
ref = new B(); // geçerli, çünkü B extends A
```

Burada ref değişkeninin statik tipi A'dır. Ancak gösterdiği nesnenin gerçek tipi çalışma zamanında değişebilir.

```
Java
```

```
ref.draw();
```

Bu çağrı yapıldığında çalışma zamanı sistemi şu soruyu sorar: ref şu anda hangi gerçek nesneyi gösteriyor?

Derleme Zamanı Tipi:

ref : A

Çalışma Zamanı Durumu 1:

ref → A nesnesi

draw() → A.draw()

Çalışma Zamanı Durumu 2:

ref → B nesnesi

draw() → B.draw()

Örnek

- Bir sistemde kullanılmış arabaların katalogu vardır.
- Genel bir Car sınıfı bulunur.
- Katalogdaki her özel araba için Car sınıfından türetilmiş subclass'lar olabilir.
- Her subclass ilgili arabanın resmini ve özel bilgilerini içerir.
- Kullanıcı katalogda gezer.
- İlgilendiği arabaları seçer.
- Sistem seçilen arabaların bilgilerini yazdırır.

```
<> Java

class Car {
    void display() {
        System.out.println("Generic car information");
    }

    void printInfo() {
        System.out.println("Generic car printout");
    }
}

class ToyotaCorolla2018 extends Car {
    @Override
    void display() {
        System.out.println("Toyota Corolla 2018 image and details");
    }

    @Override
    void printInfo() {
        System.out.println("Printing Toyota Corolla 2018 details");
    }
}
```

Örnek

Java

```
Car[] selectedCars = new Car[3];

selectedCars[0] = new ToyotaCorolla2018();
selectedCars[1] = new HondaCivic2020();
selectedCars[2] = new FordFocus2017();

for (Car c : selectedCars) {
    c.printInfo();
}
```

- Burada dizinin tipi Car[] olmasına rağmen her nesnenin kendi printInfo() metodu çalışır.
- Yeni bir araba eklemek için yalnızca yeni bir subclass yazılabilir:

Java

```
class BMW320i2021 extends Car {
    @Override
    void printInfo() {
        System.out.println("Printing BMW 320i 2021 details");
    }
}
```

Abstract Method

- Bazı sınıflar inheritance hierarchy içinde o kadar yukarıdadır ki, bu sınıflardan doğrudan nesne üretmek anlamlı değildir. Böyle sınıflar genellikle **abstract class** olarak tasarlanır.
- Building çok genel bir kavramdır. Doğrudan “bir Building çiz” demek çoğu zaman yeterli değildir. Çünkü hangi tür bina çizileceği belli değildir.
- Fakat bütün alt sınıfların bir draw() metoduna sahip olması istenir.

`</>` Java

```
abstract class Building {  
    abstract void draw();  
}
```

- Gövdesi olmayan, yalnızca protokolü belirtilen metoda abstract method denir.
- Burada draw() metodunun ne yapacağı tanımlanmamıştır. Sadece şu sözleşme verilmiştir:
- Building sınıfından türeyen her somut sınıf bir draw() metodu sağlamalıdır.

Concrete Subclass

```
</> Java

class FrenchGothic extends Building {
    @Override
    void draw() {
        System.out.println("Drawing a French Gothic building");
    }
}
```

- Bu sınıf artık nesne oluşturulabilir hâle gelir çünkü abstract method'u implemente etmiştir.

Abstract Class

- **Abstract Class Özellikleri**
- Bir sınıf en az bir abstract method içeriyorsa genellikle abstract class olur.
- Abstract class:
- Genellikle doğrudan instantiate edilemez.
- Alt sınıflar için ortak arayüz sağlar.
- Bazı metotların alt sınıflarda tanımlanmasını zorunlu kılar.
- Inheritance hierarchy için tasarım iskeleti sağlar.

`// C++`

```
class Building {  
public:  
    virtual void draw() = 0;  
};
```

C++'ta abstract method'a genellikle **pure virtual method** denir.

Saf Nesne Modeli

- Temel soru dildeki her şey nesne mi olmalıdır?
- Saf OOP modelde:
 - Integer nesnedir.
 - String nesnedir.
 - Array nesnedir.
 - Class nesnedir.
 - Hatta bütün yazılım sistemi nesnelerden oluşur.
 - Bütün hesaplama message passing ile yapılır.
 - Bu modele Smalltalk güçlü bir örnektir.
- Her şey nesne olduğunda dil daha uniform olur.
- Örneğin:

```
</> smalltalk  
  
5 factorial
```
- Burada 5 sayısı bile bir nesnedir ve ona factorial mesajı gönderilir.
- Bu yaklaşım kavramsal olarak iyidir.

Saf Nesne Modeli

- Basit işlemler bile message passing ile yapılırsa, bu işlemler imperative dillere göre daha yavaş olabilir.
- Örneğin imperative modelde:

```
</> C
```

```
x = a + b;
```

- Bu işlem doğrudan makine seviyesinde basit bir toplama komutuna dönüşebilir.
- Saf OOP modelde ise toplama bile mesaj gönderimi olarak düşünülebilir:
- a'ya + mesajı gönder, parametre olarak b ver
- Bu daha esnek ama daha maliyetli olabilir.

Primitive Tipler + Objects

- Bazı diller primitive tipleri nesne
vanmaz Örneğin Java'da:

```
// Java
```

```
int x = 5;  
Integer y = 5;
```

- Burada:
 - int primitive type'tır.
 - Integer wrapper class'tır.
 - Bu tasarım performansı artırır ama dili daha karmaşık yapar.

- Primitive değerlerin nesne gibi kullanılabilmesi için wrapper class'lar gerekir

```
// Java
```

```
Integer x = Integer.valueOf(5);  
String s = x.toString();
```

- Burada primitive bir değer, nesne biçimine dönüştürülür.
- Bu, dil tasarımında şu trade-off'u gösterir:
 - Uniformity and purity ↔ Efficiency

Subtype

- Her subclass, parent class'ın subtype'ı mıdır?
 - Bu soru basit görünür; fakat dil tasarımı açısından çok önemlidir.
- **“is-a” ilişkisi**
 - Bir subclass, parent class'ın subtype'ı ise şu ilişki geçerli olmalıdır:
 - B is an A
 - Örneğin:
 - Truck is a Vehicle
Dog is an Animal
Waterfowl is a Bird
 - Bu durumda Truck nesnesi, Vehicle beklenen yerde kullanılabilmelidir.

Subtype

- Bir derived class gerçekten subtype ise:
 - Parent class'ın public üyelerini desteklemelidir.
 - Parent class'ın kullanılabildiği yerde kullanılabilmelidir.
 - Type error oluşturmamalıdır.
 - Davranışsal olarak parent class ile uyumlu olmalıdır.
- Bu, daha sonra yazılım mühendisliğinde **Liskov Substitution Principle** olarak da tartışılır.

- Ada Subtype Örneği

```
ada  
  
subtype Small_Int is Integer range -100..100;
```

- Small_Int:
 - Integer işlemlerine sahiptir.
 - Ancak yalnızca -100..100 aralığındaki değerleri tutabilir.
 - Integer beklenen yerlerde kullanılabilir.

Subclass Farklılaşması

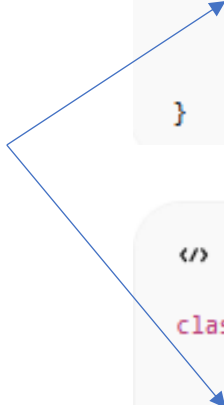
- Bir subclass parent class'tan birçok biçimde farklı olabilir:
 - Yeni method ekleyebilir.
 - Bazı method'ları kaldırabilir.
 - Method parametre tiplerini değiştirebilir.
 - Return type değiştirebilir.
 - Method gövdesini değiştirebilir.
 - Public üyeleri private hâle getirebilir.
- Fakat her fark subtype uyumluluğu açısından güvenli değildir.

Override Kısıtı

- Bir subclass'ın override ettiği method, parent class'taki method ile uyumlu olmalıdır.
- Bu bazı dillerde geçerlidir çünkü Integer, Number ile uyumludur.
- Ama parametre sayısını değiştirmek override değil, overload olabilir:

```
</> Java  
  
class A {  
    Number getValue() {  
        return 10;  
    }  
}
```

```
</> Java  
  
class B extends A {  
    @Override  
    Integer getValue() {  
        return 20;  
    }  
}
```



Override Kısıtı

- Ama parametre sayısını değiştirmek override değil, overload olabilir.
- Bu draw() metodunu override etmez; farklı imzaya sahip yeni bir method tanımlar.

`<>` Java

```
class B extends A {  
    void draw(int size) {  
        System.out.println("Drawing with size");  
    }  
}
```

Çoklu Miras (Multiple Inheritance)

- Dil tasarımında önemli sorulardan biri şudur:
- Dil yalnızca single inheritance mı desteklemelidir, yoksa multiple inheritance da desteklenmeli midir?
- **Single Inheritance**
 - Bir sınıf yalnızca bir parent class'tan miras alır.
 - Single inheritance daha basit, anlaşılır ve yönetilebilir bir yapı sağlar.

`</> Java`

```
class Animal {  
}  
  
class Bird extends Animal {  
}  
  
class Waterfowl extends Bird {  
}
```

```
Animal  
|  
Bird  
|  
Waterfowl
```

Çoklu Miras (Multiple Inheritance)

- **Multiple Inheritance**
 - Bir sınıf iki veya daha fazla parent class'tan miras alır.
- Burada MultiFunctionDevice, hem Printer hem de Scanner davranışlarını miras alır.
- Bazı kavramlar gerçek dünyada birden fazla kategorinin özelliklerini taşıyabilir.
- Bir teaching assistant hem student hem de employee özelliklerine sahip olabilir.

`// C++`

```
class Printer {  
public:  
    void print();  
};
```

```
class Scanner {  
public:  
    void scan();  
};
```

```
class MultiFunctionDevice : public Printer, public Scanner {  
};
```

TeachingAssistant

├─ Student

└─ Employee

Çoklu Miras Problemi

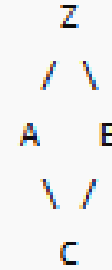
- Eğer iki parent class aynı isimli method içerirse belirsizlik oluşabilir.
- Şu çağrı belirsizdir:
 - C c;
c.display();
 - // A.display mı, B.display mı?
- Bu durumda programcı hangi parent class'taki method'un çağrılacağını belirtmek zorunda kalabilir.

⟨⟩ C++

```
class A {  
public:  
    void display() {  
        cout << "A display";  
    }  
};  
  
class B {  
public:  
    void display() {  
        cout << "B display";  
    }  
};  
  
class C : public A, public B {  
};
```


Elmas Miras Kavramı

- Multiple inheritance ile ortaya çıkan en önemli sorunlardan biri **diamond inheritance** veya **shared inheritance** problemidir.
- Burada:
 - A, Z sınıfından türemiştir.
 - B, Z sınıfından türemiştir.
 - C, hem A hem de B sınıfından türemiştir.
- Sorun:
 - C, Z sınıfından gelen üyeleri bir kez mi, iki kez mi miras almalıdır?



Elmas Miras Kavramı

- Bu durumda C içinde teorik olarak iki farklı sum olabilir:
- C object
 - A üzerinden gelen Z.sum
 - B üzerinden gelen Z.sum
- Şu ifade belirsiz olabilir:

```
</> C++  
  
C c;  
c.sum = 10; // Hangi sum?
```

```
</> C++  
  
class Z {  
public:  
    int sum;  
};  
  
class A : public Z {  
};  
  
class B : public Z {  
};  
  
class C : public A, public B {  
};
```

Elmas Miras Kavramı

- C++ gibi dillerde diamond problem için virtual inheritance kullanılabilir.
- Bu durumda C, Z kısmını tek kopya olarak miras alır.

```
</> C++  
  
class Z {  
public:  
    int sum;  
};  
  
class A : virtual public Z {  
};  
  
class B : virtual public Z {  
};  
  
class C : public A, public B {  
};
```

Çoklu Miras

- Multiple inheritance sınıf organizasyonunu karmaşıktırabilir.
- Tasarım yapmak zorlaşabilir.
- Bakım maliyeti artabilir.
- Sınıflar arası bağımlılıklar çoğalabilir.
- Bu nedenle bazı dil tasarımcıları multiple inheritance'ı doğrudan desteklememeyi tercih eder.

Çoklu Miras

- Örneğin Java'da kullanılan Interfaces, multiple inheritance'ın bazı avantajlarını sağlar; fakat daha az problem üretir.

`</>` Java

```
interface Printable {  
    void print();  
}  
  
interface Scannable {  
    void scan();  
}  
  
class MultiFunctionDevice implements Printable, Scannable {  
    public void print() {  
        System.out.println("Printing");  
    }  
  
    public void scan() {  
        System.out.println("Scanning");  
    }  
}
```

Object Allocation

- Object allocation ve deallocation için iki önemli tasarım sorusu vardır:
 - Nesneler nereden allocate edilir?
 - Heap'te oluşturulan nesneler nasıl deallocate edilir?
- Bir dilde nesneler şu alanlarda oluşturulabilir:

Allocation Yeri	Açıklama
Stack	Otomatik yaşam süresi, hızlı erişim
Heap	Dinamik yaşam süresi, reference/pointer ile erişim

Heap-Dynamic Objects

- Eğer bütün nesneler heap'te oluşturulursa:

```
</> Java
```

```
A a = new A();  
B b = new B();
```

```
</> Java
```

```
A a;  
B b = new B();  
  
a = b; // sadece referans atanır
```

- Bu durumda değişkenler nesnenin kendisini değil, nesneye referansı tutar.
- Avantajları:
 - Nesne ataması pointer/reference ataması olur.
 - Subtype assignment kolaylaşır.
 - Farklı boyuttaki subclass nesneleri aynı referans tipiyle gösterilebilir.

Stack-Dynamic Objects

- C++ gibi dillerde nesneler stack üzerinde doğrudan değer olarak tutulabilir.

```
// C++
```

```
class A {  
public:  
    int x;  
};  
  
class B : public A {  
public:  
    int y;  
};
```

```
// C++
```

```
B b1;  
A a1;  
  
a1 = b1;
```

- Burada b1, A kısmı ve B'ye özgü y alanını içerir.
- Fakat a1 yalnızca A kadar alana sahiptir.
- Bu nedenle b1 nesnesinin y alanı kopyalanmaz.
- Bu duruma **object slicing** denir.

Object Slicing

B object:

```
+-----+-----+  
|  x  |  y  |  
+-----+-----+
```

A object:

```
+-----+  
|  x  |  
+-----+
```

a1 = b1 sonrası:

A object:

```
+-----+  
|  x  |  
+-----+
```

y alanı kaybolur.

Heap Allocation

- Heap'te oluşturulan nesneler için ikinci soru şudur:
 - Nesne belleği nasıl geri verilecek?
- İki temel yaklaşım vardır:
- **Explicit deallocation**
 - Programcı delete, free gibi işlemlerle belleği serbest bırakır.
 - Dangling pointer riski vardır.
- **Implicit deallocation**
 - Garbage collection kullanılır.
 - Programcı belleği doğrudan yönetmez.

Dangling Pointer Riski

```
⟨/⟩ C++
```

```
A* p = new A();
```

```
delete p;
```

```
p->draw(); // tehlikeli: dangling pointer
```

p artık geçerli bir nesneyi göstermediği hâlde kullanılmaya çalışılır.

Bu, explicit deallocation'ın önemli risklerinden biridir.

Dynamic ve Static Binding

- OOP için dynamic binding önemlidir; fakat her method çağrısının dynamic olması zorunlu mudur?
- Bu bir dil tasarımı sorusudur.

Static Binding

- Method çağrısının hangi metoda bağlanacağı derleme zamanında bellidir.
- Avantajı:
 - Daha hızlıdır.
 - Runtime dispatch maliyeti yoktur.
 - Derleyici optimizasyonları daha kolaydır.

```
// Java
```

```
final class MathUtil {  
    static int square(int x) {  
        return x * x;  
    }  
}
```

Bu tür çağrılar genellikle dynamic dispatch gerektirmez.

Dynamic Binding

- Method çağrısının hangi metoda bağlanacağı çalışma zamanında belirlenir.

Java

```
Animal a = new Dog();  
a.speak();
```

Burada speak() çağrısı runtime'da Dog.speak() metoduna bağlanır.

Avantajı:

- Polymorphism sağlar.
- Genişletilebilirliği artırır.
- Subclass davranışlarını destekler.

Dezavantajı:

- Static binding'e göre daha maliyetlidir.
- Runtime method lookup gerekir.

Nested Class

- **Nested class**, bir sınıfın başka bir sınıf içinde tanımlanmasıdır.
- Temel motivasyon Information hiding.
- Eğer bir sınıf yalnızca başka bir sınıf tarafından kullanılacaksa, onu dışarıda görünür yapmak gereksizdir.

```
<> Java

class CarCatalog {
    private class CatalogEntry {
        String model;
        int year;
        double price;
    }

    private CatalogEntry[] entries;
}
```

Burada CatalogEntry yalnızca CarCatalog içinde anlamlıdır. Dışarıdan görünmesi gerekmez.

Object Initialization

- Nesne oluşturulduğunda başlangıç değerleri nasıl atanacaktır?
- Bu da önemli bir dil tasarımı sorusudur.
- Burada Truck oluşturulurken önce parent class olan Vehicle kısmı initialize edilir.

`</>` Java

```
class Vehicle {  
    int year;  
    String make;  
  
    Vehicle(int year, String make) {  
        this.year = year;  
        this.make = make;  
    }  
}  
  
class Truck extends Vehicle {  
    int wheels;  
  
    Truck(int year, String make, int wheels) {  
        super(year, make);  
        this.wheels = wheels;  
    }  
}
```


Subclass Initialization

- Bir subclass nesnesi oluşturulduğunda şu işlemler gerekir:
 - Vehicle kısmı initialize edilir.
 - Truck kısmı initialize edilir.
 - Nesne kullanılabilir hâle gelir.
- Dil tasarımındaki soru şudur:
- Parent class initialization otomatik mi yapılır, yoksa programcı açıkça mı belirtmelidir?
- Java'da `super(...)` çağrısı ile parent constructor çalıştırılır.

Java

```
Truck t = new Truck(2020, "Volvo", 6);
```